Sistema de Gestión para Clínica Privada

Especificación funcional y de diseño

Jezabel Rosso · Juana Ruiz Luque

Octubre de 2026

1. Resumen

Aplicación de escritorio para gestionar la atención en consultorios externos de una clínica privada: registro de pacientes, emisión de órdenes de consulta con número de turno correlativo, y (en diseño) el registro de diagnósticos y prácticas por parte de los profesionales y los reportes administrativos de cierre de turno.

La aplicación está desarrollada en **Java 11 con JavaFX**, persiste en **SQLite** vía JDBC y se organiza en capas (modelo, DAO, DTO y controladores).

Estado actual: el módulo de **Recepción** está implementado y funcional. Los módulos de **Médico** y **Administración** están modelados y prototipados, pero no implementados.

2. Contexto del problema

En la clínica, los turnos de consultorios externos se entregan **por orden de llegada**, sin reserva previa. El paciente se acerca al mostrador, indica el servicio que necesita (clínica médica, pediatría, radiología, etc.) y su número de documento. Recepción verifica si está registrado, lo da de alta si hace falta y emite una **orden de consulta** con el profesional elegido, el costo según el tipo de consulta y un número de turno.

El objetivo del sistema es reemplazar el circuito en papel por uno digital y construir una **historia clínica digital** con los diagnósticos y prácticas de cada paciente, por profesional y por fecha. Al cierre de cada turno de trabajo (mañana 06:30–13:00, tarde 13:00–20:00, noche 20:00–24:00) se emiten el listado de caja y las planillas de consultas y prácticas.

3. Alcance y actores

Actor	Responsabilidades	Estado
Recepción	Consultar y registrar pacientes, generar órdenes de consulta, asignar turno	Implementado
Profesional (médico)	Consultar sus órdenes, registrar diagnóstico y prácticas realizadas	Diseñado
Administración	Listado de caja por turno y resumen de consultas	Diseñado

4. Requisitos funcionales

Los casos de uso se documentaron con planillas completas (curso normal, cursos alternativos, pre y postcondiciones). Este es el resumen:

N°	Caso de uso	Actor	Estado
UC-001	Consultar registro de paciente por DNI	Recepción	Implementado
UC-002	Registrar nuevo paciente	Recepción	Implementado

UC-003	Gestionar orden de consulta	Recepción	Implementado
UC-004	Asignar turno (incluido en UC-003)	Recepción	Implementado
UC-005	Consultar orden de consulta	Profesional	Diseñado
UC-006	Registrar diagnóstico del paciente	Profesional	Diseñado
UC-007	Registrar práctica realizada	Profesional	Diseñado
UC-008	Generar listado de caja por turno	Administración	Diseñado
UC-009	Generar resumen de consultas	Administración	Diseñado

Además, la aplicación incluye una pantalla de **gestión de turnos** que lista los turnos emitidos junto con paciente, DNI y servicio.

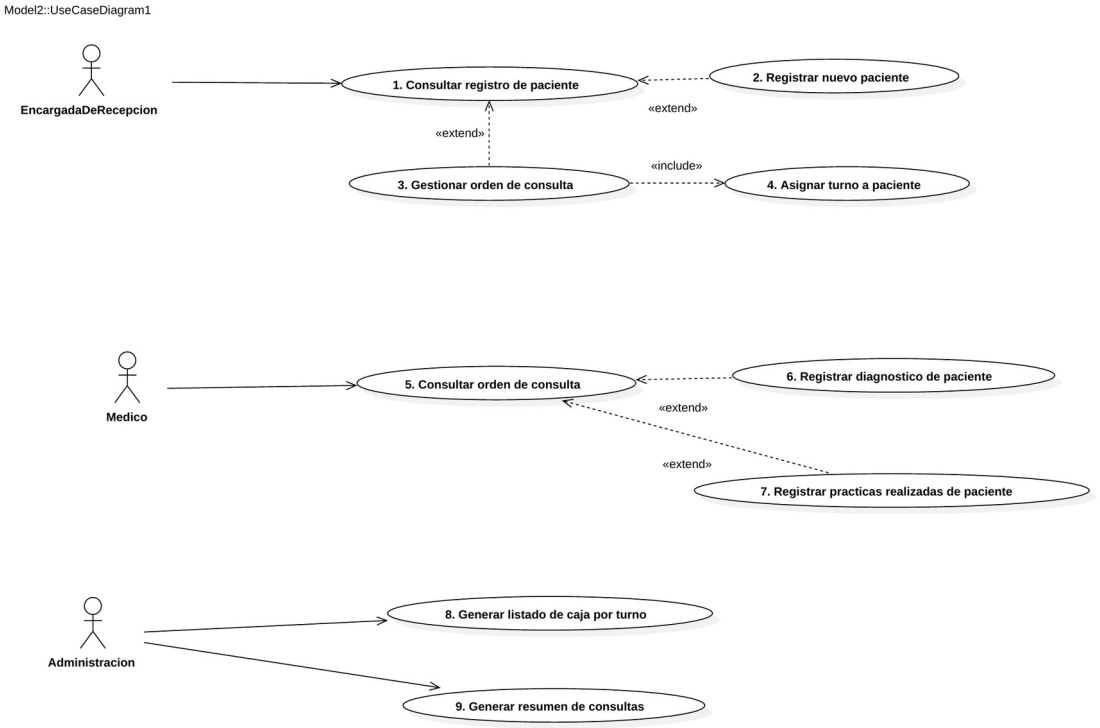


Diagrama de casos de uso

5. Reglas de negocio

Código	Regla	Estado
RN-01	Los turnos se otorgan por orden de llegada, sin reserva.	Implementado
RN-02	Antes de emitir una orden, el paciente debe estar registrado (se busca por DNI). Si no existe, la interfaz ofrece registrarlo.	Implementado
RN-03	Un paciente menor de 18 años requiere indicar el DNI de un jefe de grupo familiar que ya figure en el listado de pacientes.	Implementado
RN-04	Cada servicio tiene tipos de consulta asociados, cada uno con su costo; el médico se elige entre los del servicio seleccionado.	Implementado
RN-05	El número de turno es correlativo dentro de cada día (único por fecha y número).	Implementado
RN-06	La historia clínica registra diagnósticos y prácticas por paciente, con fecha, profesional y observaciones.	Modelado
RN-07	Al finalizar cada turno de trabajo se emite el listado de caja y las planillas de consultas y prácticas.	Diseñado

6. Modelo de dominio

El dominio gira en torno a la **Persona** (con uno o más domicilios), que puede ser **Paciente**, **Médico**, **Recepcionista** o **Administrativo**. El paciente tiene una **historia clínica** con sus detalles (diagnóstico, fecha y observaciones) y puede depender de un **jefe de grupo familiar**, que es otro paciente.

Una **orden de consulta** vincula un turno, un paciente, un médico y un servicio. Cada **servicio** se relaciona de forma muchos a muchos con sus **tipos de consulta**, que determinan el costo. El **listado de caja** agrupa las órdenes de un turno de trabajo y calcula el total.

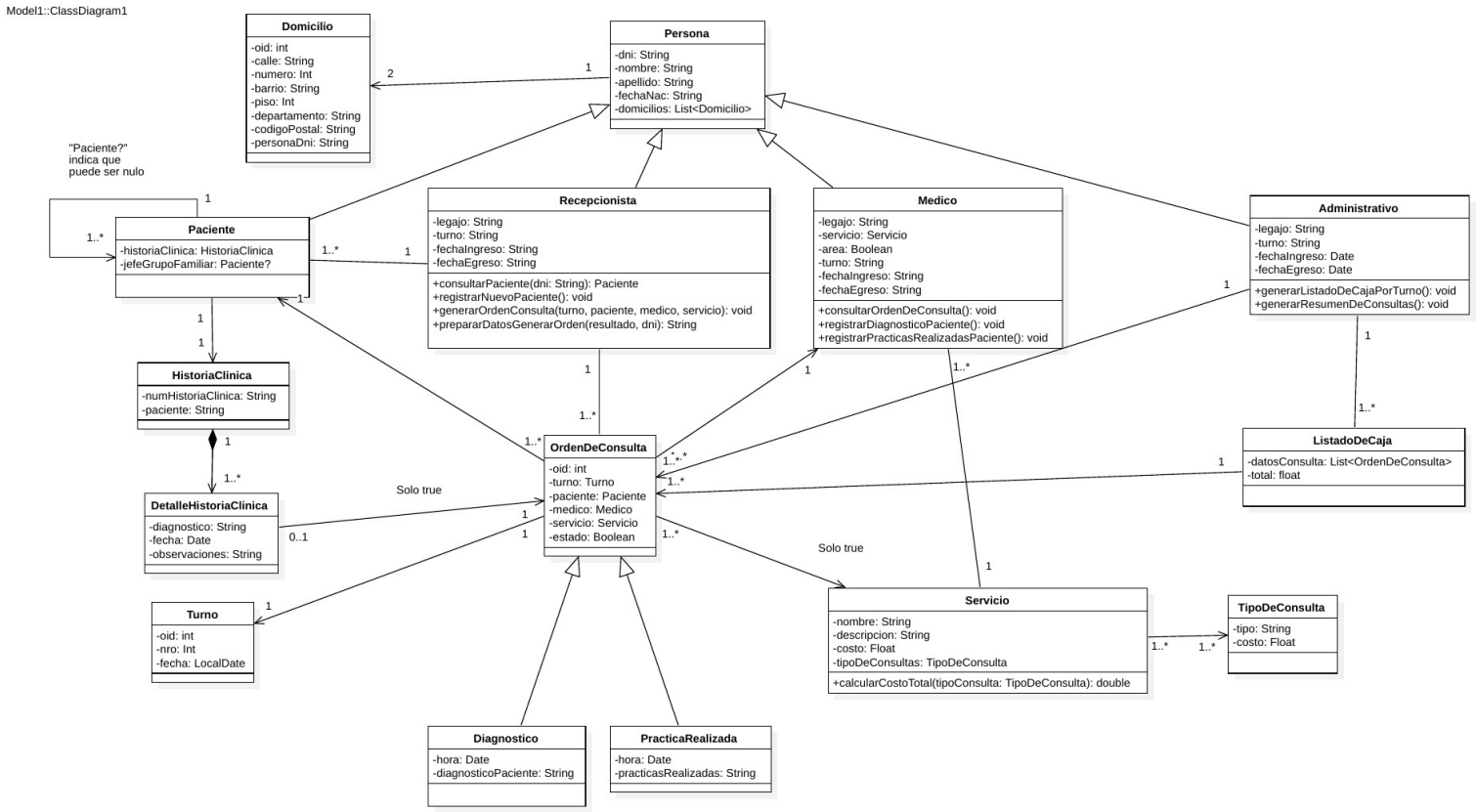
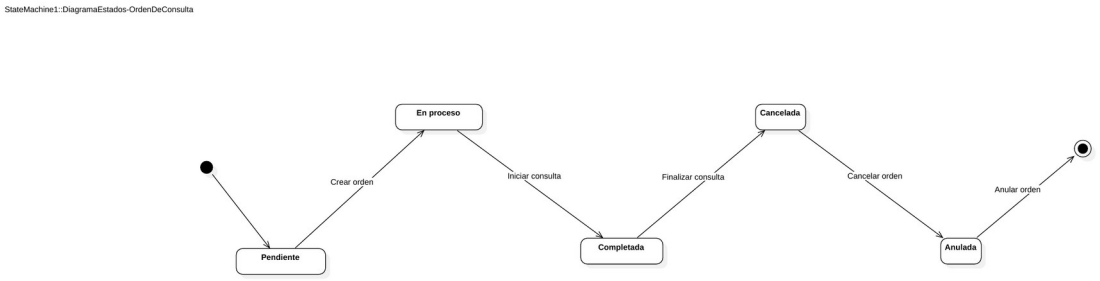


Diagrama de clases del dominio

El ciclo de vida de una orden se modeló como una máquina de estados con los estados *Pendiente*, *En proceso*, *Completada*, *Cancelada* y *Anulada*.



Máquina de estados de la orden de consulta

7. Arquitectura

La aplicación sigue una arquitectura en capas con separación entre interfaz, lógica de aplicación y acceso a datos:

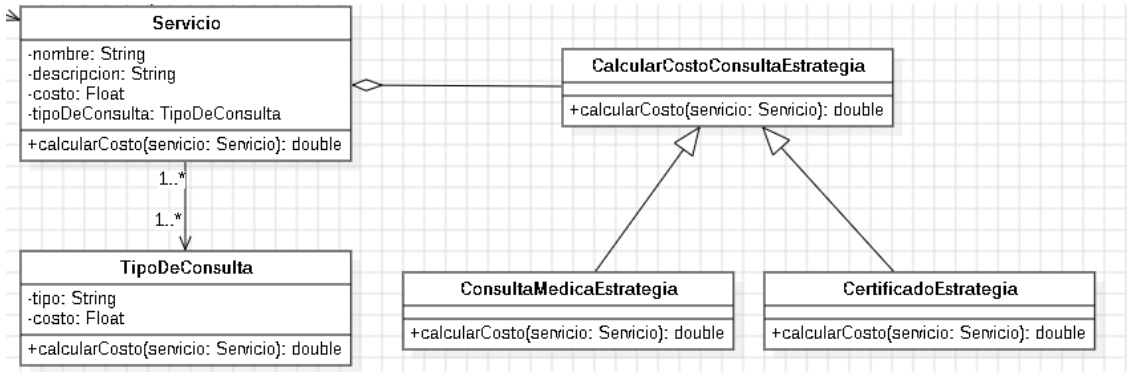
Capa	Paquete	Responsabilidad
Vista	`resources/*.fxml`, `styles.css`	Cinco pantallas FXML con estilos propios
Controladores	`controlador`	Eventos de la interfaz, validaciones y coordinación con los DAO
Modelo	`modelo`	Entidades del dominio y reglas de negocio
DTO	`dto`	Objetos de transporte entre capas (por ejemplo `PacienteCompletoDto`, `TurnoDetalleDto`)
Acceso a datos	`dao`	Consultas SQL con JDBC y `PreparedStatement`

Decisiones de diseño:

- **DAO genérico.** La interfaz `Dao<T>` define las operaciones comunes (`listarTodos`, `insertar`, `modificar`, `borrar`, `buscar`, `listarPorCriterio`) y obtiene la conexión por defecto desde `ConexionSql`.
- **Singleton para la conexión.** `ConexionSql` mantiene una única instancia y reabre la conexión si se cerró.
- **DTO para datos combinados.** `PacienteCompletoDto` reúne datos personales, domicilio, historia clínica y jefe de grupo familiar, que viven en tablas distintas, para mostrarlos en una sola tabla de la interfaz.
- **Costo de la consulta.** `Servicio.calcularCostoTotal` suma el costo base del servicio y el costo del tipo de consulta elegido.
- **Numeración de turnos.** `TurnoDao.generarNuevoTurno` toma el último número del día y le suma uno; la restricción `UNIQUE (nro, fecha)` en la base evita duplicados.

8. Patrón de diseño: Estrategia (Strategy)

El diseño aplica el patrón **Estrategia** como parte fundamental del sistema de servicios médicos, para manejar las distintas formas de cálculo de costos y descuentos aplicables a los servicios que ofrece la clínica.



Patrón Estrategia aplicado al cálculo del costo de consulta

Cómo está planteado

- `Servicio` conoce su costo base y el `TipoDeConsulta` asociado (con su propio costo), y delega el cálculo en una estrategia.
- `CalcularCostoConsultaEstrategia` es la interfaz común: define la operación `calcularCosto(servicio)`, que devuelve el costo en `double`.
- Hay una implementación por cada tipo de consulta: `ConsultaMedicaEstrategia` y `CertificadoEstrategia`, cada una con su regla de cálculo.

Qué problema resuelve

- Evita condicionales por tipo de consulta dentro de `Servicio`: cada regla de cálculo vive en su propia clase.
- Permite incorporar una nueva forma de cobro o un descuento agregando una clase, sin modificar las existentes.
- Alinea el código con la regla de negocio: la consulta médica y el certificado de buena salud tienen costos distintos aun siendo del mismo servicio.

Estado: el patrón está definido en el diseño. La versión actual del código calcula el costo con `Servicio.calcularCostoTotal`, que suma el costo del servicio y el del tipo de consulta; llevar la estrategia a código es uno de los próximos pasos.

9. Modelo de datos

Base SQLite (`dbdoo.db`) con 16 tablas:

Grupo	Tablas
Personas	`Persona`, `Domicilio`, `Paciente`, `Medico`, `Recepcionista`, `Administrativo`
Historia clínica	`HistoriaClinica`, `DetalleHistoriaClinica`, `Diagnostico`, `PracticaRealizada`
Servicios	`Servicio`, `TipoDeConsulta`, `ServicioTipoDeConsulta` (relación N:M)
Atención	`Turno`, `OrdenDeConsulta`
Administración	`ListadoDeCaja`

Claves destacadas: `Persona` se identifica por DNI; `Paciente` se identifica por número de historia clínica y se autorrelaciona (`jefeGrupoFamiliar`); `Turno` tiene la restricción única (`nro, fecha`).

10. Flujo principal de Recepción

1. La recepcionista ingresa el **DNI** del paciente en la pantalla principal.
2. Si el paciente **existe**, se habilita *Generar orden*. Si **no existe**, se habilita *Registrar nuevo paciente*.
3. En el registro se cargan datos personales y domicilio. Si el paciente es menor de 18 años, se valida que el jefe de grupo familiar exista.

4. En *Generar orden* se elige el **servicio**. La pantalla carga los tipos de consulta y los médicos de ese servicio, y muestra el **costo total** al elegir el tipo de consulta.
5. Al guardar, el sistema crea el **turno del día** con número correlativo, inserta la orden y confirma con el número de turno asignado.
6. La pantalla *Turnos* permite consultar los turnos emitidos.

11. Prototipos de interfaz

Los prototipos de todo el sistema se diseñaron en Figma antes de implementar. Los de Recepción se corresponden con las pantallas desarrolladas; los de Médico y Administración corresponden a los módulos pendientes.

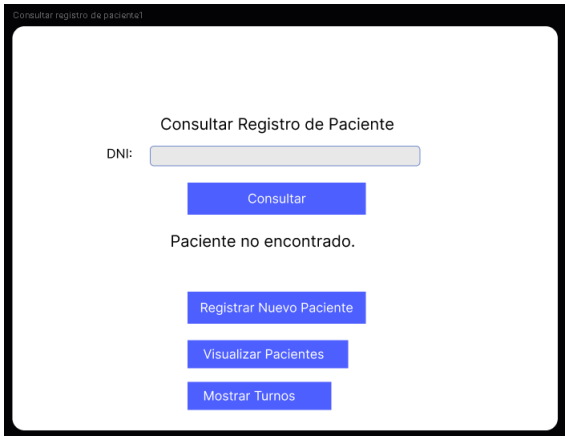
Recepción



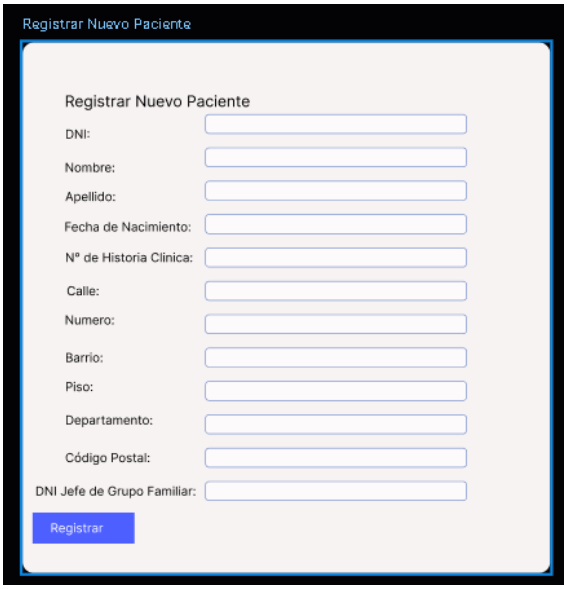
Consulta de paciente



Paciente encontrado



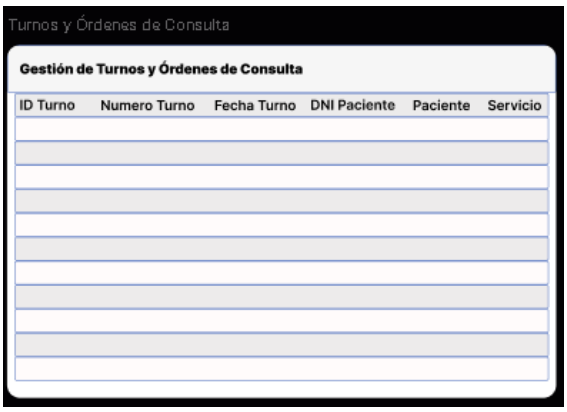
Paciente no encontrado



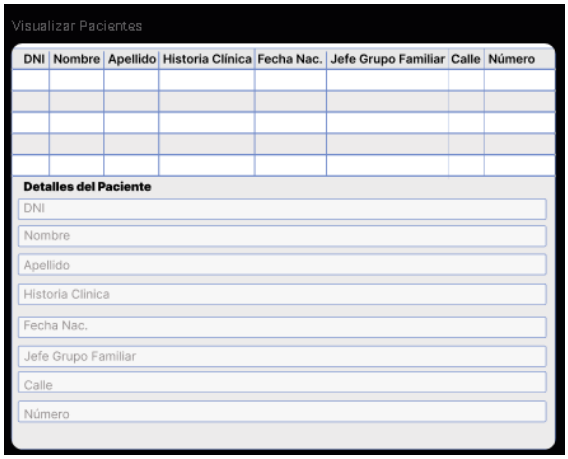
Registro de paciente



Generar orden de consulta

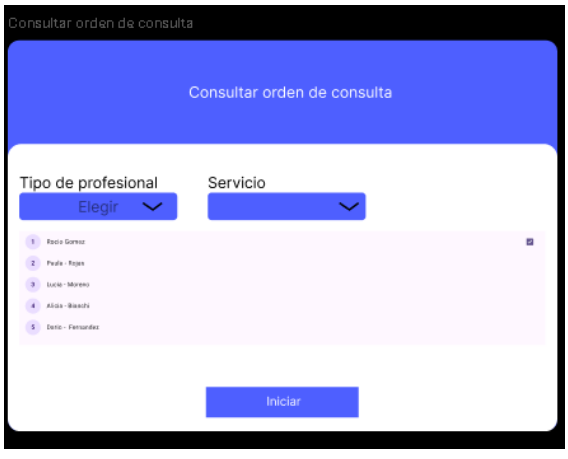


Gestión de turnos



Detalle del paciente

Médico



Órdenes asignadas

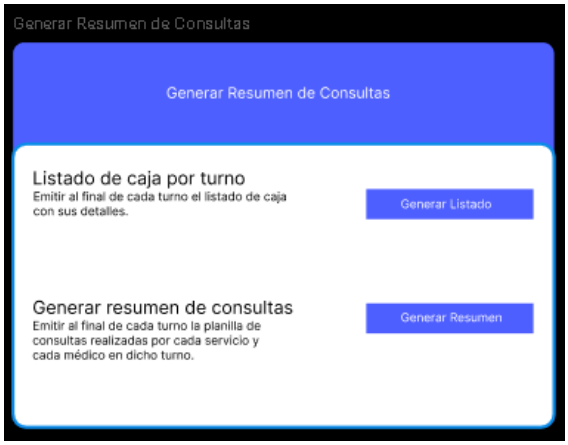


Registro de diagnóstico



Registro de práctica realizada

Administración



Listado de caja y resumen de consultas

Prototipo completo en Figma: https://www.figma.com/design/Gee4DfrmVPRVEAYkiG0B4f/Untitled?node-id=175-442&t=KrLAKTMeTKfvWqo4-1

12. Tecnologías y ejecución

Componente	Detalle
Lenguaje	Java 11
Interfaz	JavaFX 13 (FXML + CSS)
Persistencia	SQLite con `sqlite-jdbc` 3.34.0, acceso por JDBC
Construcción	Maven (plugin `javafx-maven-plugin`)
Pruebas	JUnit 4, TestFX y AssertJ configurados en el `pom.xml`
Modelado	UML (casos de uso, clases, colaboración, secuencia y estados)

Ejecución: con JDK 11 o superior y Maven instalados, ubicarse en la carpeta `clinicaPrivadaDOO` y ejecutar:

```
mvn javafx:run
```

La aplicación busca el archivo `dbdoo.db` en el directorio desde el que se ejecuta, por eso debe lanzarse desde esa carpeta.

13. Estado y próximos pasos

Implementado: consulta y alta de pacientes (con validación de jefe de grupo familiar), generación de órdenes con cálculo de costo y número de turno correlativo, y pantalla de turnos.

Próximos pasos, por prioridad:

1. **Pruebas de unidad.** El `pom.xml` ya incluye JUnit, TestFX y AssertJ, pero no hay tests escritos. Los candidatos naturales son `Servicio.calcularCostoTotal`, la numeración de turnos y la validación de menores.
2. **Módulo Médico (UC-005 a UC-007).** Consultar órdenes por servicio y registrar diagnóstico y prácticas.
3. **Módulo Administración (UC-008 y UC-009).** Listado de caja por turno de trabajo y planillas de consultas.
4. **Patrón Strategy para el cálculo de costos.** Hoy el costo es una suma fija; llevar a código la estrategia del diseño permitiría incorporar descuentos o tarifas por cobertura sin tocar `Servicio`.
5. **Estados de la orden.** El campo `estado` se guarda como booleano; la máquina de estados de cinco estados está solo en el diseño.
6. **Consistencia del esquema.** La clave foránea de `ListadoDeCaja` apunta a una tabla antigua (`OrdenDeConsulta_old`), y `Diagnostico` y `PracticaRealizada` se relacionan con el paciente pero no con la orden.
7. **Regla de costo.** La descripción del negocio asocia el costo al tipo de consulta (por ejemplo, consulta médica y certificado tienen precios distintos), mientras que la implementación suma además un costo base por servicio. Conviene unificar el criterio y los datos.
8. **Transacciones.** Solo `MedicoDao` y `RecepcionistaDao` manejan transacciones explícitas; el alta de un paciente involucra varias tablas y es candidata a hacerlo.